Healthcare Content Generator

Course Name: GenAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01.	Vanshika Vyas	EN22CS3011062
02.	Tulsi Chouhan	EN22CS3011038
03.	Ujjawal Goswami	EN22CS3011043
04.	Swayam Mishra	EN22CS3011009
05.	Mukund Mishra	EN22ME304064

Group Name:02D9

Project Number:GAI-02

Industry Mentor Name: Mr.Rishabh Tripathi

University Mentor Name:Prof. Vineeta Rathore

Academic Year:2025-26

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features *(Just mention key features here no need to go into details)*
2. Overall Architecture / Workflow
3. Tools & Technologies Used (*If applicable)*

Results & Output

Add the below details here:

1. *Screenshots / outputs*
2. *Reports / dashboards / models*
3. *Key outcomes*

Conclusion

Mention a brief conclusion about your project summing up everything you have worked on and the key learning.

Future Scope & Enhancements

Problem Statement & Objectives

1. **Problem Statement :** The healthcare industry heavily relies on **manual documentation** for creating patient summaries, clinical notes, and treatment reports. This process is often **time-consuming, repetitive, and prone to human errors**, especially when dealing with large volumes of patient data.

   Healthcare professionals must ensure that documentation follows **standard medical terminology, proper formatting, and accuracy**, which further increases their workload. Additionally, generating context-aware and structured content requires **significant expertise and effort**.

   There is a lack of intelligent systems that can:
   - Automatically generate **high-quality medical documents**
   - Maintain **consistency and standardization**
   - Utilize **contextual medical knowledge**
   - Reduce **time and manual effort**

2. **Project Objectives :** The primary objective of the Healthcare Content Generator is to develop an AI-powered system that automates the generation of structured and high-quality medical documentation using Generative AI techniques.
   Key Objectives:
   - **Automate Healthcare Documentation**
     To reduce manual effort by automatically generating patient summaries, clinical notes, and treatment reports.
   - **Ensure Accuracy and Consistency**
     To maintain standardized medical terminology, proper formatting, and professional tone in all generated content.
   - **Leverage Generative AI (LLM)**
     To utilize Large Language Models for intelligent and context-aware content generation.

- **Implement RAG (Retrieval-Augmented Generation)**
  To enhance output quality by retrieving relevant medical context from a vector database.

- **Use Vector Embeddings for Semantic Search**
  To enable meaningful and accurate retrieval of medical knowledge based on user input.

- **Design a Scalable and Modular System**
  To build a system that can handle multiple users and large datasets efficiently.

- **Improve Efficiency and Productivity**
  To help healthcare professionals save time and focus more on patient care instead of documentation.

3. **Scope of the Project :** The Healthcare Content Generator project focuses on designing and developing an AI-powered system that automates the creation of structured medical content using Generative AI and Retrieval-Augmented Generation (RAG).

In-Scope Functionalities:

- **Healthcare Content Generation :**
  The system generates medical documents such as:
  - Patient summaries
  - Clinical notes
  - Treatment reports
- **User Input Processing**
  Accepts healthcare-related topics or patient conditions and processes them for further analysis.

- **AI-Based Content Generation**
  Uses Large Language Models (LLMs) to generate context-aware and professional medical content.
- **RAG-Based Context Retrieval**
  Retrieves relevant medical information from a vector database to improve accuracy and relevance.
- **Vector Database Integration**
  Stores and retrieves embeddings using tools like FAISS, Pinecone, or ChromaDB.
- **Prompt Engineering**
  Applies structured prompts to ensure consistency, accuracy, and proper formatting.
- **Output Formatting**
  Generates well-structured output with headings, sections, and medical terminology.
- **Backend Development (Python)**
  Implements core logic, API integration, and data processing using Python.

Proposed Solution :

1.Key features :

- AI-powered healthcare content generation
- Automated patient summary and clinical note creation
- Context-aware output using RAG
- Integration with Large Language Models (LLM API)
- Vector database for semantic search
- Embedding-based input processing
- Structured and formatted document generation
- Prompt engineering for consistent output
- Scalable and modular architecture
- Fast and efficient response system
- REST API-based backend
- Option to export generated content

2.Overall Architecture / Workflow :

The system consists of the following layers:

Frontend Layer

- Provides user interface for input and output
- Accepts healthcare-related queries

Application (Backend) Layer

- Built using Python
- Handles input validation, preprocessing, and business logic

Embedding Layer

- Converts input text into vector embeddings
- Captures semantic meaning of input

Vector Database Layer

- Stores medical knowledge as embeddings
- Performs similarity-based search

Retrieval Layer (RAG)

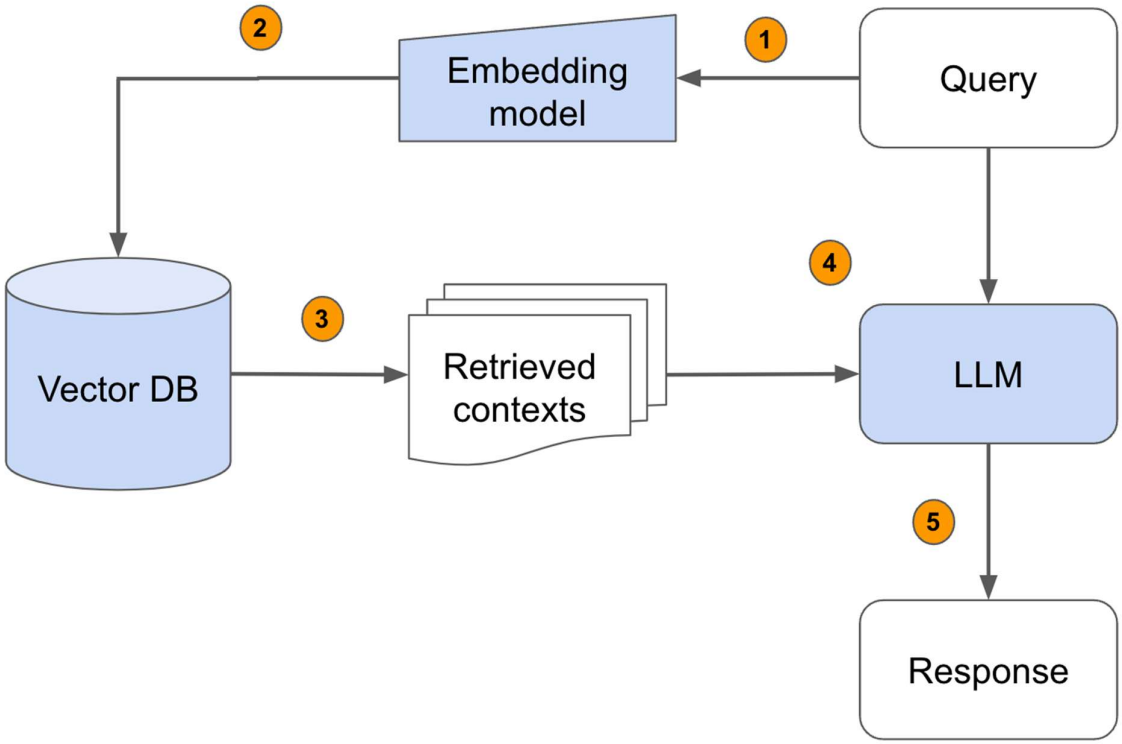
- Fetches relevant context from vector database
- Enhances accuracy of generated output

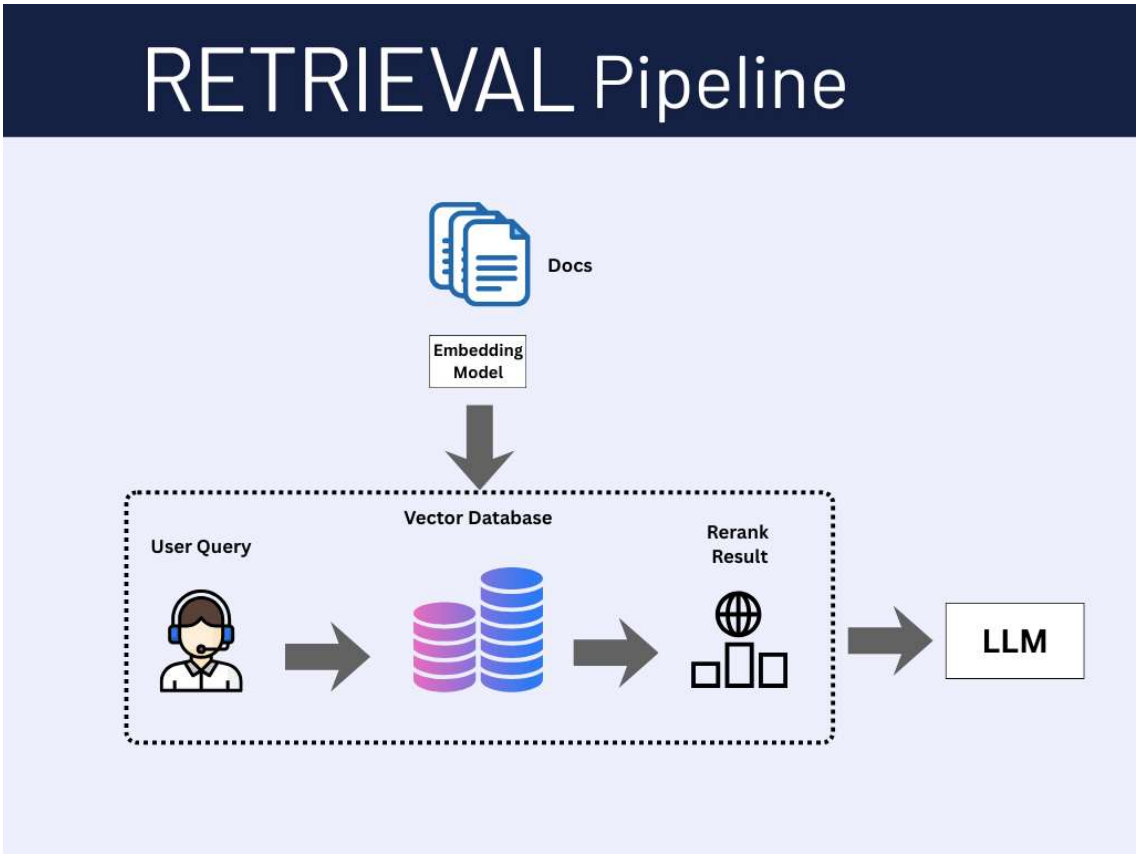
AI Layer (LLM API)

- Processes structured prompt
- Generates context-aware healthcare content

Output Layer

- Formats generated content
- Displays or exports final result


2
Embedding model
1
Query
4
Vector DB
3
Retrieved contexts
LLM
5
Response


RETRIEVAL Pipeline
Docs
Embedding Model
Vector Database
User Query
Rerank Result
LLM

Workflow (Step-by-Step Process)

1. User enters healthcare topic or query
2. Input is validated and preprocessed
3. Input converted into embeddings
4. Vector database retrieves relevant context
5. Prompt is generated using input + context
6. Prompt sent to LLM API
7. LLM generates structured content
8. Output is formatted properly
9. Final content displayed to user

3.Tools & Technologies Used :

Programming Language

- **Python** – Used for backend development, data processing, and API integration

Generative AI & LLM

- **LLM API (GPT-based models)** – For generating healthcare content
- **Prompt Engineering** – To structure and control output format

Vector Database

- **ChromaDB** – For storing and retrieving embeddings using semantic search

Embedding Models

- **Text Embedding Models** – Convert input text into vector representations for similarity search

Backend Framework & APIs

- **REST APIs** – For communication between components
- **FlaskAPI** – For building backend services

RAG (Retrieval-Augmented Generation)

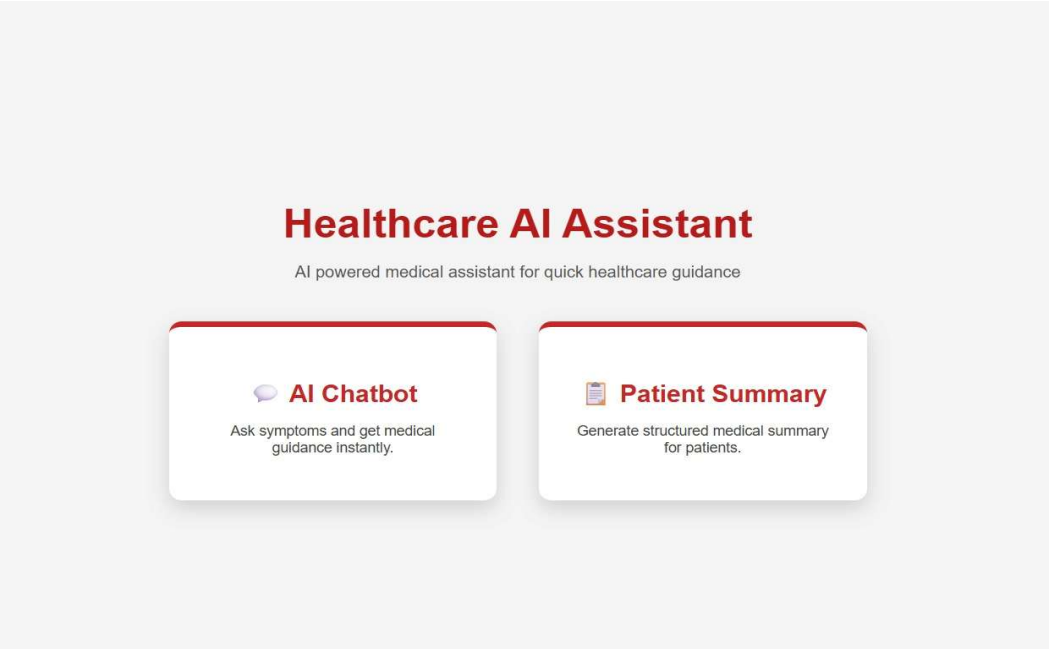
- Combines LLM with vector database to improve accuracy and context-awareness

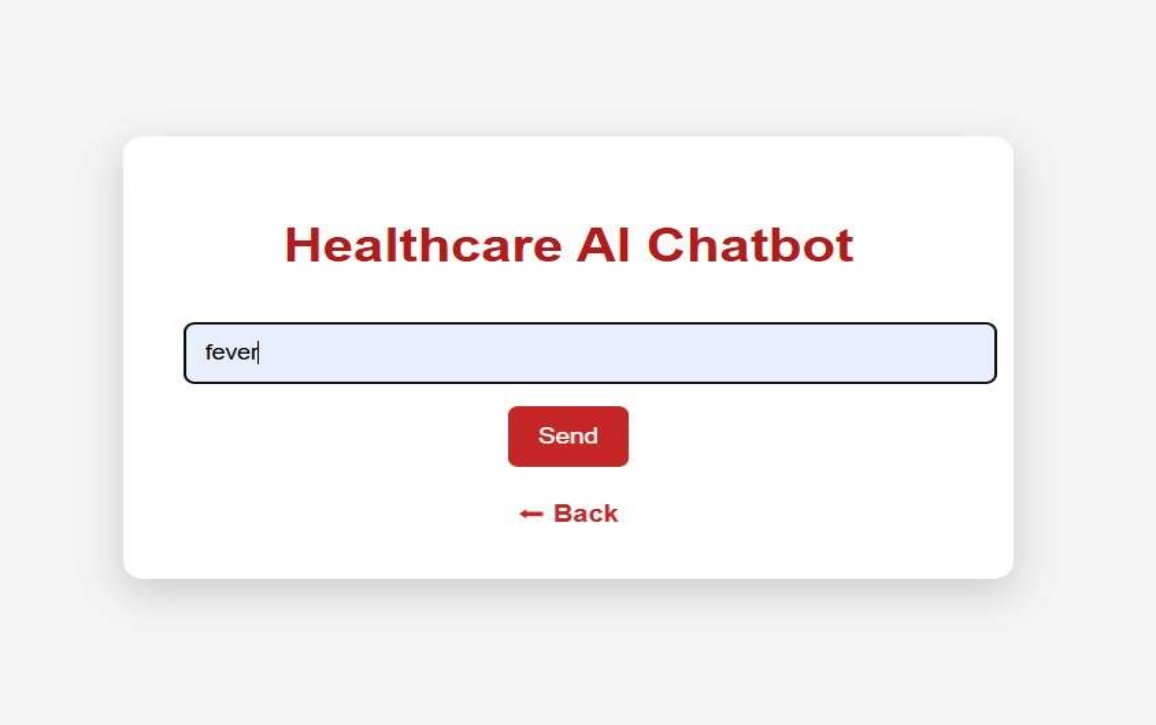
Development Tools

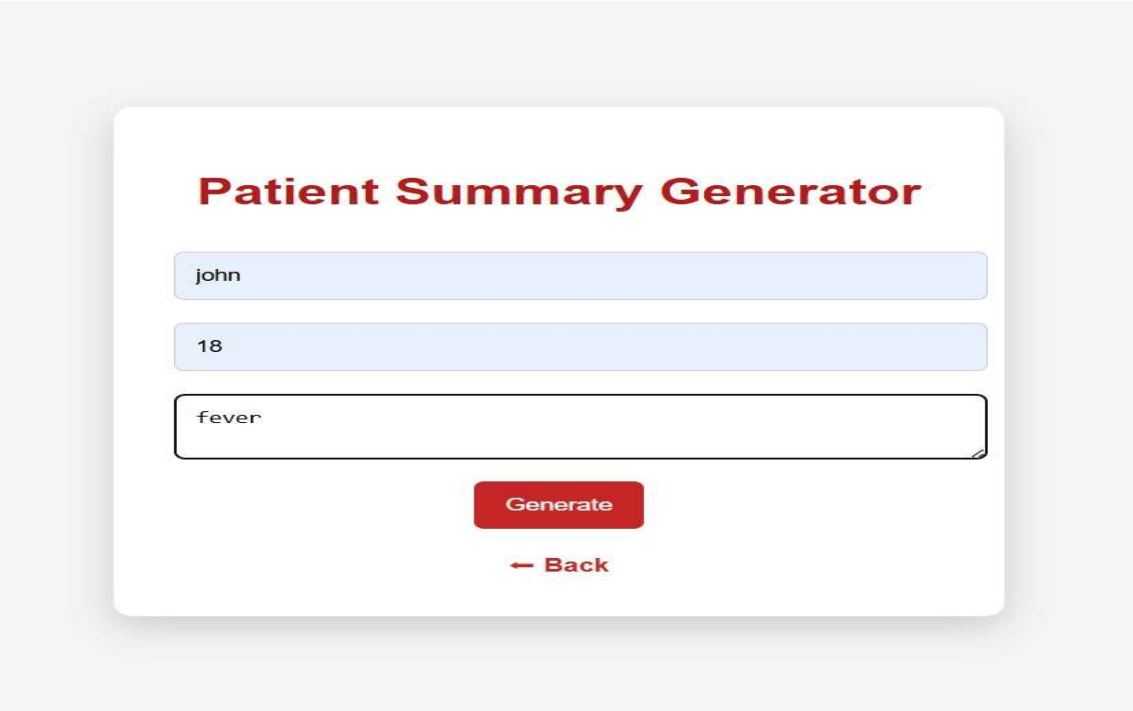
- **VS Code** – Development environment
- **Git & GitHub** – Version control and project management

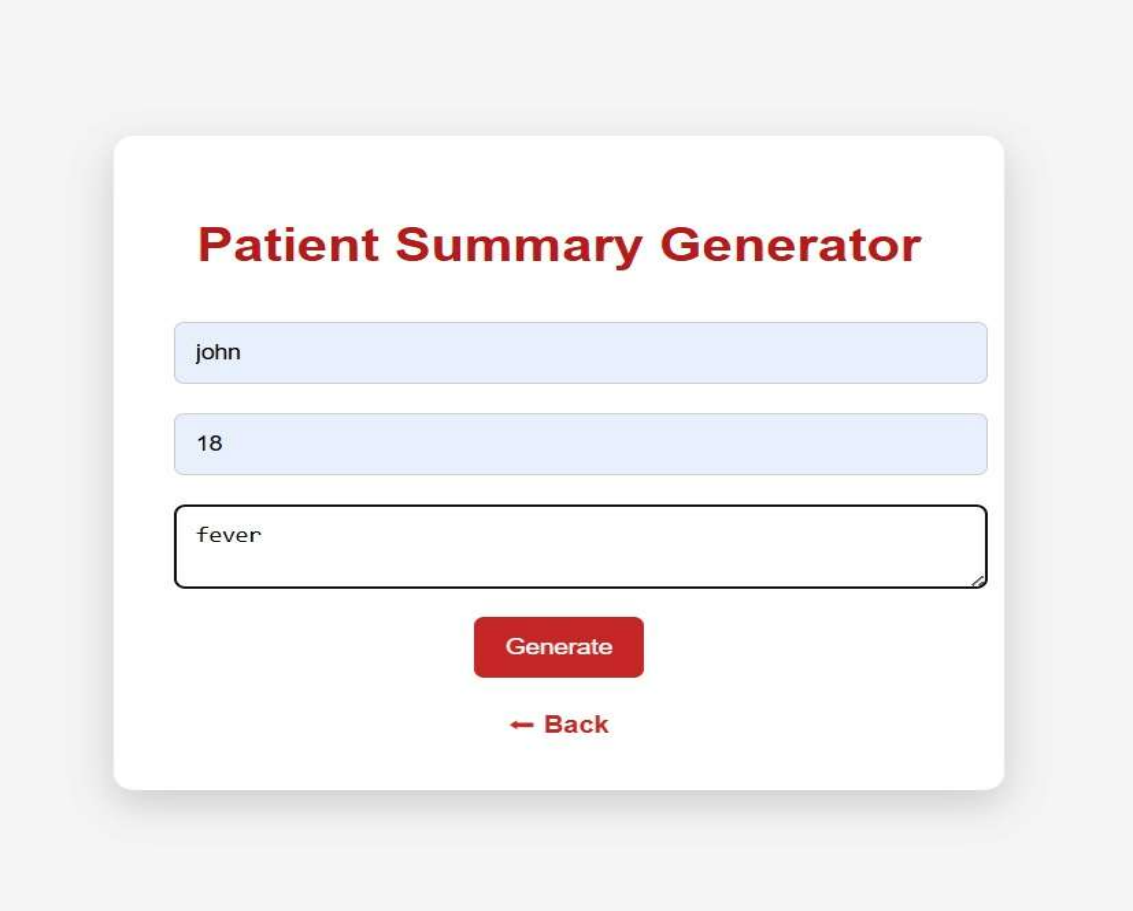
Results & Output

1. ***Screenshots / outputs :***






Patient Summary Generator
john
18
fever
Generate
← Back


Patient Summary Generator
john
18
fever
Generate
← Back

Patient Summary Generator

Patient Name

Age

Enter symptoms

Generate

Generated Summary

Patient Summary

Name: john

Age: 18

Symptoms:

fever

Observation:

The symptoms indicate a possible mild illness or infection.

Advice:

Patient should consult a healthcare professional for proper diagnosis.

← Back

2. **Reports / dashboards / models :**

   a. **Generated Healthcare Reports :**

   Description:

   The system generates detailed and structured healthcare reports such as:

   - Patient Summaries
   - Clinical Notes
   - Treatment Reports

   Each report includes:

   - Patient condition
   - Symptoms
   - Diagnosis
   - Treatment plan
   - Recommendations

   b. **Dashboard :**

   Description:

   Although not the primary focus, a basic dashboard can be implemented to monitor:

   - Number of requests processed
   - API response time
   - System performance
   - Usage statistics

   c. **AI Models Used :**

   Description:

   The system utilizes the following AI models:

   - Large Language Model (LLM)
     Used for generating healthcare content (GPT-based models)

- embedding Model
  Converts text into vector form for semantic understanding
- RAG Model (Retrieval-Augmented Generation)
  Combines retrieved context with LLM for accurate output

3. **Key outcomes :**

- Automated generation of healthcare documents such as patient summaries and clinical notes
- Reduction in manual documentation effort and time
- Generation of structured and well-formatted medical content
- Improved accuracy using Retrieval-Augmented Generation (RAG)
- Successful integration of LLM API, embeddings, and vector database
- Context-aware content generation using semantic search
- Scalable and modular system architecture
- Faster response and efficient processing
- Practical implementation of Generative AI concepts
- Development of a complete end-to-end AI-based system

Conclusion

The Healthcare Content Generator project demonstrates the effective use of Generative AI and Retrieval-Augmented Generation (RAG) in automating healthcare documentation. The system successfully converts basic user input into structured, accurate, and professionally formatted medical content such as patient summaries and clinical notes.

Future Scope & Enhancements

- Integration with real hospital systems such as Electronic Health Records (EHR)
- Support for real-time patient data and live updates
- Implementation of advanced medical knowledge bases for improved accuracy
- Addition of multilingual support for wider usability
- Development of a user-friendly and interactive frontend interface